

A Practical Preimage Attack on the 19-Round SHA-256 Compression Function via a Global σ_0 -Difference Table

Kameldip Singh Basra

Independent Researcher

kameldipbasra@gmail.com

ORCID: 0009-0001-0977-4614

September 2026

Abstract

We present an algebraic candidate-generation method for SHA-256 compression reduced to 19 of 64 rounds. The standard initial value and feed-forward are retained, all 256 target bits are fixed, and the input is an unrestricted 512-bit block rather than a padded message. From the target, a backward chain fixes eight state words. Each attempt chooses seven context words, sweeps one 32-bit state word, and enforces three message-schedule constraints with a reusable 16-GiB table for $u \mapsto \sigma_0(u) - u$. Two constraints form a fixed-point iteration; an exact final test checks the provisional treatment of W_9 . A complete sweep took about 25 seconds on one NVIDIA H100 with one initial chain and about 89 seconds with the four chains used in the campaign.

We publish seven forward-verified examples: six for digests generated from random messages and one for the fixed all-ones benchmark. Zaikin’s SAT-based 19-round result on that benchmark predates this work. Three dedicated runs succeeded in their 4th, 21st and 35th contexts. A preregistered stratified campaign processed 240 full contexts: 3 of 120 selected by a cheap screening rule produced a preimage, while 0 of 120 controls did. Pooling the arms gives 3/240, but does not estimate a uniformly sampled context. Screening raised an intermediate low-16-bit counter by a trajectory-counted factor 3.41 (identity-level bounds 2.42–3.44); its effect on final success remains unresolved. With only three campaign events, the data do not determine a mean runtime. A newer chosen-context paper supersedes these performance figures and gives computed 20-round examples. Neither paper affects full SHA-256.

Keywords: SHA-256, preimage attack, step-reduced, message schedule, table lookup, GPU cryptanalysis.

1 Introduction

SHA-256 [3] is the most widely deployed member of the SHA-2 family. Its compression function has 64 rounds, and the security margin of the full function is usually assessed through attacks on step-reduced variants.

Attack model. Let F_R denote the eight-word state after the first R SHA-256 rounds, before feed-forward, and let IV be the standard initial value. Given a 256-bit digest h , a *preimage* in this paper is a 512-bit block $(W_0, \dots, W_{15})$ with

$$h = \text{IV} + F_R(\text{IV}, W_0, \dots, W_{15}).$$

This is the standard preimage setting for the compression function: the chaining value is fixed, nothing about any original message is known, and only the digest is given. It is not a pseudo-preimage or free-start setting, in which the attacker would also choose the chaining value. The 16 words of the block are unconstrained, so the result concerns the compression function and not the length-padded hash. For a single-block 55-byte message the padding would constrain parts of W_{13}, W_{14}, W_{15} . Although the repository contains an exploratory closed-form way to impose those word values, no padded preimage was obtained; we make no padded-hash claim.

Prior work. Theoretical preimage attacks on step-reduced SHA-256 reach far beyond 19 rounds, but at costs within a few bits of the generic 2^{256} : Isobe and Shibutani [4] give a one-block preimage attack on 24 steps at 2^{240} ; Aoki et al. [5] reach 43 steps at $2^{254.9}$ by meet-in-the-middle; Khovratovich, Rechberger and Savelieva [7] reach 45 steps at $2^{255.5}$ with bicliques; Guo, Li, Liu, Wang and Zhang [6] have since extended the meet-in-the-middle line to 46 steps at $2^{254.5}$ and 47 steps at $2^{255.6}$. None of these has been executed in full. Collision attacks are a different problem and are far more advanced: Li, Liu and Wang [10] lowered the 31-step collision attack on SHA-256 to $2^{49.8}$ and Li et al. [11] then computed a practical 31-step collision, both building on local-collision techniques [8, 9]; they do not yield preimages.

The practical preimage frontier belongs to SAT-based cryptanalysis, which Mironov and Zhang [12] applied early to hash functions, in the collision setting. Davydov, Pikhtovnikov, Kiryanova and Zaikin [13] find preimages of the 17- and 18-round compression function with Kissat under six CNF encodings (CBMC, Transalg, and four adder variants of SAT-encoding), report 19 rounds as unsolved under every one of them, and reach 19 rounds only in a weakened form in which at most 31 of the 256 output bits are constrained, and they also report 19-round collisions, a different problem; their published benchmark instances target the all-zero digest. For SHA-256 both literatures use “round” and “step” for the same 64 applications of the step function, so no prior result changes its round count under the other convention. Zaikin [14] then constructs intermediate inverse problems between consecutive round counts, parameterizes a CDCL solver on them, and reports that the resulting parallel Cube-and-Conquer solver inverted the 29-step MD5, 24-round SHA-1 and 19-round SHA-256 compression functions for the first time. For SHA-256 the target is one fixed digest, the all-ones string, with all 256 bits fixed, in the same model as here: compression function, standard IV, no padding, feed-forward retained. The 19-round instance itself was not solved within seven days on 192 cores; the preimage was obtained from the intermediate problem between rounds 18 and 19 whose solutions are 19-round preimages with probability $1/8$, in 18 h 33 min on 192 CPU cores (two 96-core AMD EPYC 9654), about 148 core-days, and it verifies against the reference implementation. That article was submitted in March 2025 and appeared in January 2026. Priority at 19 rounds is Zaikin’s and we claim none.

This work. We give an algebraic, table-based preimage attack on the 19-round compression function. After a one-time 16-GiB precomputation, each attempt is a single 2^{32} sweep on one GPU: about 25 seconds on the recorded H100 with one seed chain, or 89 seconds with the four chains used in the campaign. The campaign was stratified. Its selected arm produced 3/120 successes and its control arm 0/120; the pooled 3/240 rate describes that campaign but not a uniform draw from the context population. A plug-in population-weighted point estimate, $0.3(3/120) + 0.7(0/120) = 7.5 \times 10^{-3}$, corresponds to about 3.3 GPU-hours, but only three total events make this estimate highly uncertain. We therefore report observed work and rates rather than a precise mean runtime. The ingredients are:

1. a backward chain that recovers the eight state words $a_{11}, \dots, a_{18}$ from the digest alone (§3);

2. a global table inverting $u \mapsto \sigma_0(u) - u$, built once and valid for every target and every attempt (§4);
3. the observation that in each of the three schedule constraints one unknown state word appears with opposite signs in two places, so that the constraint reduces to a single table lookup once the remaining unknowns are provisionally fixed (§5); the provisional assignment is closed by a fixed-point iteration and tested exactly by a 32-bit consistency check;
4. seven forward-verified preimages: six of digests of random messages, three from dedicated runs and three from a preregistered 240-attempt campaign that records arm-specific success counts, and one of the all-ones digest that Zaikin attacked, found in 84 attempts (§6, §7);
5. a measured, preregistered account of how much a cheap screening pass improves the intermediate yield, and of what it does not establish (§5.7).

A companion note [2] records finite negative results for a direct 20-round extension. A later chosen-context construction [1] supersedes this paper’s performance figures and gives two computed 20-round examples; see the note added at the end. Appendix A records an exact cancellation identity in the message schedule found along the way, together with the reason the search experiments built on it are not claimed as results.

2 SHA-256 Preliminaries

Notation. All arithmetic is modulo 2^{32} unless stated otherwise. We use $\oplus$, $\&$, $\bar{x}$ for bitwise XOR, AND, NOT. Rotations and shifts of a 32-bit word x :

$$\text{ROTR}^n(x) = (x \gg n) \mid (x \ll (32 - n)), \quad \text{SHR}^n(x) = x \gg n.$$

Round functions.

$$\begin{aligned} \Sigma_0(a) &= \text{ROTR}^2(a) \oplus \text{ROTR}^{13}(a) \oplus \text{ROTR}^{22}(a), \\ \Sigma_1(e) &= \text{ROTR}^6(e) \oplus \text{ROTR}^{11}(e) \oplus \text{ROTR}^{25}(e), \\ \sigma_0(x) &= \text{ROTR}^7(x) \oplus \text{ROTR}^{18}(x) \oplus \text{SHR}^3(x), \\ \sigma_1(x) &= \text{ROTR}^{17}(x) \oplus \text{ROTR}^{19}(x) \oplus \text{SHR}^{10}(x), \\ \text{Ch}(e, f, g) &= (e \& f) \oplus (\bar{e} \& g), \\ \text{Maj}(a, b, c) &= (a \& b) \oplus (a \& c) \oplus (b \& c). \end{aligned}$$

State labels. We write the 8-word compression state after round r as $(a_r, a_{r-1}, a_{r-2}, a_{r-3}, e_r, e_{r-1}, e_{r-2}, e_{r-3})$, with IV boundary values $a_{-4}, \dots, a_{-1}$ and $e_{-4}, \dots, e_{-1}$:

$$a_{-1}=\text{IV}[0], \ a_{-2}=\text{IV}[1], \ a_{-3}=\text{IV}[2], \ a_{-4}=\text{IV}[3], \ e_{-1}=\text{IV}[4], \ e_{-2}=\text{IV}[5], \ e_{-3}=\text{IV}[6], \ e_{-4}=\text{IV}[7].$$

Round equations.

$$T_1^{(r)} = \Sigma_1(e_{r-1}) + \text{Ch}(e_{r-1}, e_{r-2}, e_{r-3}) + e_{r-4} + K_r + W_r, \tag{1}$$

$$T_2^{(r)} = \Sigma_0(a_{r-1}) + \text{Maj}(a_{r-1}, a_{r-2}, a_{r-3}), \tag{2}$$

$$a_r = T_1^{(r)} + T_2^{(r)}, \tag{3}$$

$$e_r = a_{r-4} + T_1^{(r)}. \tag{4}$$

Note that e_{r-4} in (1) is the *h-register* word (boundary value $e_{-4} = \text{IV}[7]$), while a_{r-4} in (4) is the *d-register* word (boundary value $a_{-4} = \text{IV}[3]$); these are distinct IV words and distinct chain values for all $r \geq 4$.

From (3)–(4): $T_1^{(r)} = a_r - T_2^{(r)}$, and rearranging (1):

$$W_r = T_1^{(r)} - \Sigma_1(e_{r-1}) - \text{Ch}(e_{r-1}, e_{r-2}, e_{r-3}) - e_{r-4} - K_r. \quad (5)$$

Message schedule. For $r \geq 16$ the message schedule expands:

$$W_r = \sigma_1(W_{r-2}) + W_{r-7} + \sigma_0(W_{r-15}) + W_{r-16}. \quad (6)$$

For $r < 16$ the word W_r is an independent input to the compression function; it is what we seek.

Hash output. After R rounds the hash is $h[i] = \text{IV}[i] + s_i$ for $i = 0, \dots, 7$, where the final state vector is $(s_0, \dots, s_7) = (a_{R-1}, a_{R-2}, a_{R-3}, a_{R-4}, e_{R-1}, e_{R-2}, e_{R-3}, e_{R-4})$.

3 Structural Analysis

3.1 Backward Chain

From the R -round hash we recover the final state directly:

$$a_{R-1}, a_{R-2}, a_{R-3}, a_{R-4}, e_{R-1}, e_{R-2}, e_{R-3}, e_{R-4}.$$

We then extend backward using $T_2^{(r)} = \Sigma_0(a_{r-1}) + \text{Maj}(a_{r-1}, a_{r-2}, a_{r-3})$, $T_1^{(r)} = a_r - T_2^{(r)}$, $a_{r-4} = e_r - T_1^{(r)}$, iterating for $r = R-1, R-2, R-3, R-4$ to recover $a_{R-5}, \dots, a_{R-8}$.

For $R = 19$ this yields $\{a_{11}, a_{12}, \dots, a_{18}\}$: the state words a_{11} through a_{18} are determined by the target hash alone, with no knowledge of any preimage.

3.2 Free Parameters and Schedule Constraints

At $R = 19$ the unconstrained parameters are $a_0, \dots, a_{10}$ (11 words). Of these, $a_4, \dots, a_{10}$ are chosen uniformly at random per attempt (*context*), forming 7 attacker-controlled free words. The remaining $a_0, \dots, a_3$ are the targets of the attack phases.

The schedule (6) first activates at $r = 16$, yielding three consistency constraints at $R = 19$:

$$W_{16} = \sigma_1(W_{14}) + W_9 + \sigma_0(W_1) + W_0, \quad (\text{C0})$$

$$W_{17} = \sigma_1(W_{15}) + W_{10} + \sigma_0(W_2) + W_1, \quad (\text{C1})$$

$$W_{18} = \sigma_1(W_{16}) + W_{11} + \sigma_0(W_3) + W_2. \quad (\text{C2})$$

Here W_{16}, W_{17}, W_{18} are recovered from the target and the context via the backward chain and (5), using the known $a_{11}, \dots, a_{18}$ together with the context values $a_4, \dots, a_{10}$ (which supply e_{12}, e_{13}, e_{14} through a_8, a_9, a_{10} ; together with e_{15}, e_{16}, e_{17} from the chain alone these are the e -words that (5) needs at $r = 16, 17, 18$). Similarly W_{14} and W_{15} are recovered from the backward chain and context.

4 The σ_0 -Difference Table

Definition 1. The σ_0 -difference table $\mathcal{T}$ is an array of 2^{32} 32-bit entries:

$$\mathcal{T}[v] = u$$

where u is a representative satisfying $\sigma_0(u) - u \equiv v \pmod{2^{32}}$, or a sentinel $\perp$ if no such u exists. When multiple u share the same v , the deterministic on-disk table stores the largest such u . The GPU build used for the runs (`build_table` in `h100_extended.py`) writes roots in ascending chunks with a CUDA scatter whose winner at repeated indices PyTorch does not specify; it stores a valid root but not necessarily the largest. Experiment E3 compares smallest- and largest-root policies on one matched sample and detects no resolved difference there; it does not establish policy invariance in general. On disk the table is an `int32` array with sentinel -1 ; the value $v = 0x20000000$, whose only root is $u = 0xFFFFFFFF$, is consequently treated as a miss. The GPU sentinel is the `int32` minimum and can analogously obscure that bit pattern if it is the selected representative. Either format can lose at most one stored domain representative, which is negligible for the reported rates.

Construction. $\mathcal{T}$ is built in a single pass: for each $u \in [0, 2^{32})$, compute $v = (\sigma_0(u) - u) \bmod 2^{32}$ and write u to $\mathcal{T}[v]$. On an NVIDIA H100 in chunks of 2^{25} , the build takes under 2 seconds on the recorded H100 and requires 16 GiB of GPU memory.

Coverage. The function $f(u) = \sigma_0(u) - u \bmod 2^{32}$ is not injective. An exhaustive construction finds 2,721,603,628 of the 2^{32} possible values v have at least one preimage, corresponding to **63.37%** coverage.

Usage. Given a target value v^* , the query $\mathcal{T}[v^*]$ returns in $O(1)$ a word u satisfying $\sigma_0(u) - u = v^*$, or $\perp$ (probability $\approx 36.6\%$).

5 The 19-Round Preimage Attack

5.1 Overview

The attack operates in three phases per context:

C0. Sweep $a_0 \in [0, 2^{32})$. A table lookup recovers W_1 (and hence a_1) for about 63.4% of a_0 values (the image fraction of the table).

C1/C2. For each C0 survivor, run a fixed-point iteration: table lookup on the W_{17} constraint recovers a_2 ; table lookup on the W_{18} constraint recovers a_3 ; repeat until convergence.

W9 filter. Test the consistency condition $\varepsilon(a_2, a_3) = 0$ of Proposition 1 exactly; its pass rate is measured in §5.6. Candidates that pass are verified by forward evaluation.

The key that makes all three phases work is a common algebraic pattern: in each constraint, the target unknown a_i appears with *opposite signs* in two different terms, causing exact cancellation in the σ_0 -difference lookup target.

5.2 Context Precomputation

Fix a target hash h . Run the backward chain to obtain $a_{11}, \dots, a_{18}$. Choose uniformly random context $a_4, \dots, a_{10}$. Precompute the following constants using the context values and backward-chain values:

$$T_2^{\text{IV}} = \Sigma_0(a_{-1}) + \text{Maj}(a_{-1}, a_{-2}, a_{-3}), \quad (7)$$

$$C_0 = -T_2^{\text{IV}} - e_{-4} - \Sigma_1(e_{-1}) - \text{Ch}(e_{-1}, e_{-2}, e_{-3}) - K_0, \quad (8)$$

$$C_{e_0} = a_{-4} - T_2^{\text{IV}}. \quad (9)$$

From these: $W_0 = a_0 + C_0$ and $e_0 = a_0 + C_{e_0}$, both linear in a_0 .

For the context rounds $r = 7, \dots, 11$, compute $T_2^{(r)}$ and $T_1^{(r)}$, and for $r = 8, \dots, 11$ also $e_r = a_{r-4} + T_1^{(r)}$, from (1)–(4) using the context $a_4, \dots, a_{10}$ together with a_{11} from the backward chain. ($e_7 = a_3 + T_1^{(7)}$ depends on the unknown a_3 and is not a context constant.) Define:

$$W_9^{\text{base}} = T_1^{(9)} - \Sigma_1(e_8) - K_9, \quad (10)$$

$$W_{11}^{\text{base}} = T_1^{(11)} - T_1^{(7)} - \Sigma_1(e_{10}) - \text{Ch}(e_{10}, e_9, e_8) - K_{11}, \quad (11)$$

$$\text{CONST}_{10} = T_1^{(10)} - \Sigma_1(e_9) - K_{10}. \quad (12)$$

Also recover $W_{14}, W_{15}, W_{16}^{\text{bc}}, W_{17}^{\text{bc}}, W_{18}^{\text{bc}}$ from (5) at rounds 14, 15, 16, 17, 18 respectively (all from the backward chain and the context).

Finally, initialise $\hat{W}_9$ with $a_2 = a_3 = 0$:

$$\hat{W}_9 = W_9^{\text{base}} - T_1^{(5)}|_{a_3=0} - \text{Ch}\left(e_8, T_1^{(7)} + 0, T_1^{(6)}|_{a_2=a_3=0}\right), \quad (13)$$

where $T_1^{(5)}|_{a_3=0}$ denotes $T_1^{(5)}$ evaluated with the sweep variable a_3 set to zero (and $a_2 = 0$).

5.3 Phase C0: The Provisional- W_9 Form

The constraint C0 involves W_9 , which depends on the unknowns a_1, a_2, a_3 . The attack does not solve C0 exactly. It solves C0 under a provisional assignment for the part of W_9 that depends on a_2, a_3 , and tests that assignment afterwards. The following proposition makes the assignment precise.

Proposition 1 (Provisional- W_9 form). *Let W_9 be the message word determined by the state through equation (5) at $r = 9$, and let $\hat{W}_9$ be the quantity (13), computed from the context with $a_2 = a_3 = 0$. Then*

$$\hat{W}_9 - W_9 = a_1 + \varepsilon(a_2, a_3) \pmod{2^{32}},$$

where

$$\varepsilon(a_2, a_3) = [T_1^{(5)} - T_1^{(5)}|_{a_2=a_3=0}] + [\text{Ch}(e_8, e_7, e_6) - \text{Ch}(e_8, T_1^{(7)}, T_1^{(6)}|_{a_2=a_3=0})]$$

depends on a_2, a_3 only through $\text{Maj}(a_4, a_3, a_2)$, $\text{Maj}(a_5, a_4, a_3)$ and the Ch arguments $e_6 = a_2 + T_1^{(6)}$ and $e_7 = a_3 + T_1^{(7)}$, and vanishes at $a_2 = a_3 = 0$.

Proof. Apply (5) at $r = 9$:

$$W_9 = \underbrace{T_1^{(9)} - \Sigma_1(e_8) - K_9}_{W_9^{\text{base}}} - e_5 - \text{Ch}(e_8, e_7, e_6).$$

$T_1^{(9)}$ and e_8 depend only on the context $a_4, \dots, a_9$, so W_9^{base} is a context constant. The identity $e_r = a_{r-4} + T_1^{(r)}$ at $r = 5, 6, 7$ gives

$$e_5 = a_1 + T_1^{(5)}, \quad (14)$$

$$e_6 = a_2 + T_1^{(6)}, \quad (15)$$

$$e_7 = a_3 + T_1^{(7)}, \quad (16)$$

where $T_1^{(7)} = a_7 - T_2^{(7)}$ is a context constant ($T_2^{(7)} = \Sigma_0(a_6) + \text{Maj}(a_6, a_5, a_4)$), $T_1^{(6)}$ depends on a_3 through $\text{Maj}(a_5, a_4, a_3)$, and $T_1^{(5)}$ depends on a_2, a_3 through $\text{Maj}(a_4, a_3, a_2)$. Substituting (14),

$$W_9 = W_9^{\text{base}} - a_1 - T_1^{(5)} - \text{Ch}(e_8, e_7, e_6),$$

while by definition (13)

$$\hat{W}_9 = W_9^{\text{base}} - T_1^{(5)}|_{a_2=a_3=0} - \text{Ch}(e_8, T_1^{(7)}, T_1^{(6)}|_{a_2=a_3=0}).$$

Subtracting gives the stated expression. The two brackets are the a_2, a_3 -dependent parts of $T_1^{(5)}$ and of the Ch term, and both are zero at $a_2 = a_3 = 0$. $\square$

The residual ε is not zero in general: on a random state it is a nonzero 32-bit word. The attack proceeds *as if* $\varepsilon = 0$. We call this the *provisional assignment*. Under it, C0 reduces to a single table lookup (Proposition 2); C1 and C2 are then solved for a_2, a_3 by iteration (§5.4); and finally $\varepsilon(a_2, a_3) = 0$ is tested exactly (§5.5). A candidate that passes the test satisfies all three constraints with the true W_9 , and forward evaluation confirms it. The attack therefore finds those preimages whose (a_2, a_3) happen to satisfy $\varepsilon = 0$. Per context this is a 2^{-32} fraction of the roughly 2^{32} preimages that the three constraints leave over $(a_0, \dots, a_3)$, i.e. about one preimage per context. Whether the sweep reaches it is a separate question: the iteration is seeded on partial consistency with $\hat{W}_9$ and converges preferentially to $\varepsilon = 0$ points, far above the 2^{-32} per fixed point that a uniform residual would give; how often is measured in §5.6. For each of the seven verified preimages $\hat{W}_9 - W_9 = a_1$ holds exactly, as it must for any solution the attack returns; for P1, $0x72c05667 - 0x6d1e5a20 = 0x05a1fc47 = a_1$.

C0 lookup. For each $a_0 \in [0, 2^{32})$, compute:

$$e_0 = a_0 + C_{e_0}, \tag{17}$$

$$W_0 = a_0 + C_0, \tag{18}$$

$$g(a_0) = -\Sigma_0(a_0) - \text{Maj}(a_0, a_{-1}, a_{-2}) - e_{-3} - \Sigma_1(e_0) - \text{Ch}(e_0, e_{-1}, e_{-2}) - K_1, \tag{19}$$

$$F_0 = W_{16}^{\text{bc}} - \sigma_1(W_{14}) - g(a_0) - \hat{W}_9 - W_0. \tag{20}$$

Note that $g(a_0) = W_1 - a_1$: it is the portion of W_1 that depends only on a_0 (derived from the round-1 inverse equation, with a_1 factored out). Then:

Proposition 2 (C0 cancellation). *Under the provisional assignment $\varepsilon(a_2, a_3) = 0$, $F_0 = \sigma_0(W_1) - W_1 \pmod{2^{32}}$.*

Proof. Under the provisional assignment, Proposition 1 gives $W_9 = \hat{W}_9 - a_1$, so from the C0 schedule constraint

$$\sigma_0(W_1) = W_{16}^{\text{bc}} - \sigma_1(W_{14}) - W_9 - W_0 = W_{16}^{\text{bc}} - \sigma_1(W_{14}) - \hat{W}_9 + a_1 - W_0.$$

Since $W_1 = a_1 + g(a_0)$:

$$\sigma_0(W_1) - W_1 = W_{16}^{\text{bc}} - \sigma_1(W_{14}) - \hat{W}_9 + a_1 - W_0 - a_1 - g(a_0) = F_0. \quad \square$$

The a_1 terms cancel exactly. A single table lookup $\mathcal{T}[F_0]$ returns W_1 , and then $a_1 = W_1 - g(a_0)$.

5.4 Phases C1/C2: Fixed-Point Iteration

For each C0 survivor (a_0, a_1, W_1) , derive:

$$e_1 = a_{-3} + a_1 - \Sigma_0(a_0) - \text{Maj}(a_0, a_{-1}, a_{-2}), \tag{21}$$

$$T_2^{(2)} = \Sigma_0(a_1) + \text{Maj}(a_1, a_0, a_{-1}), \tag{22}$$

$$F_{12} = -T_2^{(2)} - e_{-2} - \Sigma_1(e_1) - \text{Ch}(e_1, e_0, e_{-1}) - K_2. \tag{23}$$

Note that $F_{12} = W_2 - a_2$: the portion of W_2 depending only on (a_0, a_1) .

Proposition 3 (C1 and C2 cancellations). *Define the a_3 -dependent quantity:*

$$D(a_3) = (a_6 - \Sigma_0(a_5) - \text{Maj}(a_5, a_4, a_3)) + \text{Ch}(e_9, e_8, a_3 + T_1^{(7)}).$$

Let $W_{16}^{\text{sched}} = \sigma_1(W_{14}) + \hat{W}_9 + \sigma_0(W_1) + W_0$ and $W_{16}^* = W_{16}^{\text{sched}} - a_1$. Then:

$$F_{C1}(a_3) = W_{17}^{\text{bc}} - \sigma_1(W_{15}) - W_1 - \text{CONST}_{10} - F_{12} + D(a_3) = \sigma_0(W_2) - W_2, \quad (\text{C1-target})$$

$$F_{C2}(a_2) = W_{18}^{\text{bc}} - \sigma_1(W_{16}^*) - W_{11}^{\text{base}} - W_2 - F_{23}(a_0, a_1, a_2) = \sigma_0(W_3) - W_3, \quad (\text{C2-target})$$

where $F_{23}(a_0, a_1, a_2)$ is the portion of W_3 depending on (a_0, a_1, a_2) but not a_3 . Both identities are unconditional for every C0 survivor: the lookup $\mathcal{T}[F_0]$ enforces $\sigma_0(W_1) - W_1 = F_0$, and substituting the definition of F_0 gives $W_{16}^* = W_{16}^{\text{bc}}$ exactly, so $\sigma_1(W_{16}^*)$ may equally be written $\sigma_1(W_{16}^{\text{bc}})$.

Proof. C1. From (5) at $r = 10$, using $\text{CONST}_{10} = T_1^{(10)} - \Sigma_1(e_9) - K_{10}$:

$$W_{10} = \text{CONST}_{10} - e_6 - \text{Ch}(e_9, e_8, e_7).$$

By the shift-register identity $e_r = a_{r-4} + T_1^{(r)}$: $e_6 = a_2 + T_1^{(6)}(a_3)$ and $e_7 = a_3 + T_1^{(7)}$, so $\text{Ch}(e_9, e_8, e_7) = \text{Ch}(e_9, e_8, a_3 + T_1^{(7)})$. Grouping the a_3 -dependent terms into $D(a_3)$:

$$W_{10} = \text{CONST}_{10} - a_2 - D(a_3).$$

Substituting into the C1 schedule constraint $\sigma_0(W_2) = W_{17}^{\text{bc}} - \sigma_1(W_{15}) - W_{10} - W_1$:

$$\begin{aligned} \sigma_0(W_2) &= W_{17}^{\text{bc}} - \sigma_1(W_{15}) - (\text{CONST}_{10} - a_2 - D(a_3)) - W_1 \\ &= W_{17}^{\text{bc}} - \sigma_1(W_{15}) - \text{CONST}_{10} + a_2 + D(a_3) - W_1. \end{aligned}$$

Using $W_2 = a_2 + F_{12}$, the a_2 terms cancel:

$$\begin{aligned} \sigma_0(W_2) - W_2 &= (W_{17}^{\text{bc}} - \sigma_1(W_{15}) - \text{CONST}_{10} + a_2 + D(a_3) - W_1) - (a_2 + F_{12}) \\ &= W_{17}^{\text{bc}} - \sigma_1(W_{15}) - W_1 - \text{CONST}_{10} - F_{12} + D(a_3) = F_{C1}(a_3). \quad \square_{C1} \end{aligned}$$

C2. From (5) at $r = 11$: $W_{11} = T_1^{(11)} - \Sigma_1(e_{10}) - \text{Ch}(e_{10}, e_9, e_8) - e_7 - K_{11}$. Since $e_7 = a_3 + T_1^{(7)}$:

$$W_{11} = \underbrace{[T_1^{(11)} - T_1^{(7)} - \Sigma_1(e_{10}) - \text{Ch}(e_{10}, e_9, e_8) - K_{11}]}_{W_{11}^{\text{base}}} - a_3.$$

So $W_{11} = W_{11}^{\text{base}} - a_3$ with W_{11}^{base} determined by context alone.

For a C0 survivor, $F_0 = \sigma_0(W_1) - W_1$, and unwinding the definition of F_0 gives $W_{16}^{\text{bc}} = \sigma_1(W_{14}) + \hat{W}_9 + \sigma_0(W_1) + W_0 - a_1 = W_{16}^*$, so $\sigma_1(W_{16}) = \sigma_1(W_{16}^*)$ with $W_{16} = W_{16}^{\text{bc}}$ the true round-16 word. Substituting into C2: $\sigma_0(W_3) = W_{18}^{\text{bc}} - \sigma_1(W_{16}^*) - W_{11} - W_2 = W_{18}^{\text{bc}} - \sigma_1(W_{16}^*) - (W_{11}^{\text{base}} - a_3) - W_2$. Using $W_3 = a_3 + F_{23}$, the a_3 terms cancel:

$$\begin{aligned} \sigma_0(W_3) - W_3 &= W_{18}^{\text{bc}} - \sigma_1(W_{16}^*) - W_{11}^{\text{base}} + a_3 - W_2 - a_3 - F_{23} \\ &= W_{18}^{\text{bc}} - \sigma_1(W_{16}^*) - W_{11}^{\text{base}} - W_2 - F_{23} = F_{C2}(a_2). \end{aligned}$$

□

Iteration. The joint C1/C2 system defines a map

$$(a_2, a_3) \mapsto (a'_2, a'_3), \quad a'_2 = \mathcal{T}[F_{C1}(a_3)] - F_{12}, \quad a'_3 = \mathcal{T}[F_{C2}(a'_2)] - F_{23}(a_0, a_1, a'_2),$$

in which the C2 lookup uses the freshly updated a'_2 (Gauss–Seidel order, as implemented in `h100_extended.py`); a step in which either lookup misses leaves the pair unchanged. We iterate from a seed $(a_2^{(0)}, a_3^{(0)})$ for up to 8 steps. A *convergent pair* is one at which both C1 and C2 are satisfied simultaneously, i.e., the map is a fixed point. Empirical convergence rate: $1.3\text{--}1.9 \times 10^{-5}$ per C0 survivor (E1, §5.6).

K-seed optimisation. Pre-computing K initial pairs (a_2, a_3) that satisfy the low-16-bit constraint $W_9^{\text{conv}}(a_2, a_3) \bmod 2^{16} = \hat{W}_9 \bmod 2^{16}$, i.e. $\varepsilon(a_2, a_3) \equiv 0 \pmod{2^{16}}$, was measured in an early pilot to yield about $3.3\times$ more W_9 lo-pass events per unit of compute than the single $(0, 0)$ start; no per-context lo-pass difference is visible between the $K_{\text{seeds}}=0$ and $K_{\text{seeds}}=4$ datasets of Table 1.

5.5 W9 Bit-Filter

After each convergent pair (a_2, a_3) is found, recompute $\hat{W}_9 - \varepsilon(a_2, a_3)$, the a_1 -free part of W_9 at the converged values:

$$W_9^{\text{conv}} = W_9^{\text{base}} - (a_5 - \Sigma_0(a_4) - \text{Maj}(a_4, a_3, a_2)) - \text{Ch}(e_8, a_3 + T_1^{(7)}, a_2 + T_1^{(6)}(a_3)). \quad (24)$$

Two sequential filters are applied:

- **Lo filter:** $(W_9^{\text{conv}} \bmod 2^{16}) = (\hat{W}_9 \bmod 2^{16})$, i.e. the low half of ε vanishes. Passes at $52\text{--}85\times$ (E1) or $37\text{--}44\times$ (E2) the uniform 2^{-16} rate (experiments defined in §5.6).
- **Hi filter:** $(W_9^{\text{conv}} \gg 16) = (\hat{W}_9 \gg 16)$, i.e. the high half vanishes too, so that both together test $\varepsilon = 0$ exactly. Passes at $12.6\text{--}13.3\times$ (E1) or $7\text{--}11\times$ (E2) the uniform rate. This filter is *not* uniform; a coarse-binned test suggests otherwise but is underpowered against a spike at a single residual (§6.3).

The combined rate is not obtained by multiplying the two marginals: the iteration is seeded on lo-consistency, so independence of the two halves cannot be assumed and has not been established (§5.6). Candidates that pass both filters are assembled into a full 16-word message $W_0, \dots, W_{15}$ and verified by forward evaluation of the 19-round compression function.

5.6 Complexity Analysis

W9 structural enhancement. Neither half of the W_9 filter is uniform. Both enhancements arise because the C1/C2 fixed-point iteration, started from W_9 -lo-consistent seed pairs, preferentially converges to fixed points that also satisfy the W_9 constraint—an effect that acts on the full 32-bit residual, not only on the low half. Experiment **E2**, a separately written CPU reimplementing measuring over 2.2×10^9 swept a_0 values per configuration, gives:

- the *lo* filter passes at $37\text{--}44\times$ the uniform 2^{-16} rate, confirming the enhancement mechanism;
- the *hi* filter passes at $7\text{--}11\times$ the uniform rate—it is *not* uniform, contrary to what a coarse-binned test suggests (§6.3).

E1 and E2 are separate campaigns and their rates are not directly comparable: E1 deduplicates fixed points and reports per-fixed-point rates from two CPU pilots, whereas E2 reports per-swept- a_0 rates from a distinct implementation on different contexts. Both find the same

Table 1: Empirical per-context statistics. E1: two deduplicated CPU pilots of 0.20 and 1.37 context-equivalents. Lo-pass counts per context: the E1 pilots, the 165-context H100 run of §6.3, and the campaign control arm of §5.7. μ rows: the 240-context campaign. Wall times: H100.

Quantity	Value
C0 survivors (table hits)	2,721,603,628 (63.37% of 2^{32})
E1 — <i>deduplicated (unique fixed points); two CPU pilots</i>	
C1/C2 fixed points	$3.6\text{--}5.1 \times 10^4$ per context ($1.3\text{--}1.9 \times 10^{-5}$ per C0 survivor)
W_9 lo-pass events per context	28 (E1), 35.0 (165-context run), 33.7 (campaign control)
W_9 lo-pass rate per fixed point	$7.9 \times 10^{-4}\text{--}1.3 \times 10^{-3}$
Structural <i>lo</i> enhancement	$52\text{--}85\times$ above 2^{-16}
W_9 hi-pass rate per fixed point	$\approx 2.0 \times 10^{-4}$
Structural <i>hi</i> enhancement	$12.6\text{--}13.3\times$ above 2^{-16}
<i>Legacy trajectory counts (each fixed point counted once per remaining iteration)</i>	
Repeat-count inflation factor	$5.5\text{--}5.9\times$ (pilots), $6.35\times$ (campaign)
μ (original model: measured lo count $\times$ uniform conditional hi)	
	$\approx 5 \times 10^{-4}$
μ (mixed campaign arms, 240 ctx)	1.25×10^{-2} Garwood CI [$2.6\cdot 10^{-3}, 3.7\cdot 10^{-2}$]
screened arm (120 ctx, $N_{LH} = 3$)	2.50×10^{-2}
control arm (120 ctx, $N_{LH} = 0$)	$< 3.1 \times 10^{-2}$ (Garwood, 95%)
μ (independence-derived projection)	$6 \times 10^{-3}\text{--}1.3 \times 10^{-2}$
Observed contexts (P3, P2, P1)	4, 21, 35 (mean 20)
Wall time per context (H100), $K_{\text{seeds}}=0$	≈ 25 s
same, $K_{\text{seeds}}=4$ (campaign)	≈ 89 s

qualitative effect — a large *lo* enhancement and a smaller but nonzero *hi* enhancement — and the residual factor of roughly 1.5 between them is not currently explained; it is within the spread expected from context-to-context variation (§5.7), which is itself large. **E1 is the headline dataset**, since it is deduplicated and feeds the μ projection. Neither E1 nor E2 measures the success rate; the preregistered campaign below does.

To test sensitivity to the lookup table’s representative policy, a third experiment (**E3**) repeated the measurement on byte-identical contexts with a table built to keep the *smallest* rather than the *largest* preimage of each target: lo enhancement $30.0\times$ (smallest) against $28.5\times$ (largest), and hi $11.1\times$ against $6.9\times$, agreeing within Poisson error on the 8 and 13 hi events observed. This matched sample detects no resolved policy effect, but does not establish that all representative policies are equivalent. E3 used its own contexts and sample, which is why its lo figure sits below E1’s range. Direct tests of lo/hi independence at reduced filter widths give $D = P(\text{both})/[P(\text{lo})P(\text{hi})] = 1.00 \pm 0.03$ at 4 bits and 1.06 ± 0.10 at 6 bits, consistent with independence at the widths where joint events can be counted.

Multiplying the measured marginals by the deduplicated fixed-point count gives a *projection* of $\mu \approx 6 \times 10^{-3}\text{--}1.3 \times 10^{-2}$ per context (one figure per pilot). Once fixed points are deduplicated the lo-pass count is 28–35 per context across the pilots, the 165-context run and the campaign control arm (Table 1), so the lo analysis stands; the correction is almost entirely the hi filter, worth about $13\times$ rather than the two orders of magnitude a naive comparison of non-deduplicated counts suggests. We label this a projection rather than a measurement for two reasons. First, it assumes the two halves are independent; since the iteration is seeded on lo-consistency, that is exactly the assumption one should not make for free. Half-width diagnostics detect no material dependence at 4 and 6 bits, but a sparse spike at the exact 16-bit value would remain invisible to them—the same limitation that makes the binned χ^2 test uninformative (§6.3). Second, the direct estimator $\hat{\mu} = N_{LH}/C$ now has events to work with, though few. The preregistered campaign of §5.7 attacked 240 contexts at full 2^{32} exposure and

recorded $N_{LH} = 3$ complete 32-bit matches, all of which verified as preimages, giving

$$\hat{\mu} = \frac{3}{240} = 1.25 \times 10^{-2} \text{ per context,} \quad 95\% \text{ Garwood interval } [2.58 \times 10^{-3}, 3.65 \times 10^{-2}].$$

Split by arm, the screened half gave $3/120 = 2.50 \times 10^{-2}$ and the control half $0/120$, i.e. $\mu < 3.1 \times 10^{-2}$ (upper limit of the two-sided 95% Garwood interval under a Poisson count model). Two cautions apply. The pooled figure mixes the two arms and the screened half is enriched by construction, so it is not an estimate for uniformly sampled contexts; the control arm is the closer analogue and it produced no events. And three events give an interval spanning a factor of 14, so this bounds the mixed-arm rate rather than pinning it. The result is numerically compatible with the 6×10^{-3} – 1.3×10^{-2} band projected before the campaign, but the stratified design and three events do not support a stronger validation claim. The projections themselves remain unstable. After adopting global fixed-point deduplication, the independence-derived projection was 1.31×10^{-2} in a 0.20 context-equivalent pilot and 5.80×10^{-3} in a 1.37 context-equivalent pilot. That variation shows the marginal-event sample is too small for a stable projection; it does not by itself establish a directional bias. An earlier figure of 4.88×10^{-2} used repeat-counted fixed points and is not comparable with either.

Two earlier sources of error are worth recording, since both inflate μ if left uncorrected. A converged lane remains at its fixed point for every subsequent iteration, so counting convergences per iteration overcounts each fixed point—measured at 5.5 – $5.9\times$ in the CPU pilots and $6.35\times$ over the 240 campaign contexts (the `fp_replay` and `fp_trajectories` columns of the campaign data). And convergences tallied per seed chain rather than per context understate the per-context count by the number of chains. The E1 fixed-point figures above are globally deduplicated on (a_0, a_1, a_2, a_3) within a context. The campaign’s downstream counters are trajectory-counted and treated separately in Section 5.7.

What the three dedicated finds do and do not establish. The dedicated runs found preimages in their 4th, 21st and 35th contexts (solver indices 3, 20 and 34), three successes in 60 attempted contexts. We present these as demonstrations on separate targets, not as a runtime estimator: three observations cannot determine the mean, and context heterogeneity means that the mean number of matches per context need not equal the probability of at least one match. Their recorded solving times were 81.7, 511.1 and 871.6 seconds, for 1,464 seconds combined with the single (0,0) seed. Across 57 completed sweeps and the three partial winning sweeps, the average was about 25.7 seconds per full-sweep equivalent. The four-seed campaign measured 89 seconds per context; these are separate configurations and both timings are retained.

5.7 Context Screening

The clustering just hypothesised is visible and predictive for the intermediate fixed-point counter. The attack samples a context and then sweeps a_0 over 2^{32} , giving every context the same budget; that is optimal only if contexts are interchangeable. Over 280 contexts, each scored on a 2^{21} sample of a_0 , the deduplicated C1/C2 fixed-point count had mean 63.4 and variance 97,082, a variance-to-mean ratio of $\approx 1,530$ against the 1.0 that a Poisson model would give, with median 12 and maximum 4,924. The top decile of contexts held 68.6% of the total yield.

Overdispersion alone is not usable, since it could be per-sample noise. The operative question is whether a cheap score measured on one a_0 sample predicts yield on an independent one. Scoring each of 392 contexts twice on independent 2^{20} samples gave a Spearman rank correlation of 0.929 in the archived run (`screening_validation.py`, seed 99001, `data_screening_validation.json`; 0.910 in an earlier unarchived run) between the two scores, so productivity is a stable property of the context and is recoverable from a short pass. Since screening costs 2^{20} against a full sweep of 2^{32} , it is $1/4096$ of a context’s cost and is nearly free: selecting the top 1% on the screening sample gave a $12.99\times$ out-of-sample *fixed-point* yield lift

for a 2.44% overhead. We stress that this is a statement about fixed points only; the attack-relevant lift is measured below and is substantially smaller. The deep tail is also unstable—the top-1% row rests on 4 contexts of 392 and moved from $12.99\times$ to $31.93\times$ between runs—so the top-decile figure ($4.98\times$ and $6.79\times$ in two runs, of which only the second is archived) is the defensible one at this level.

Does the lift survive conversion? The score counts C1/C2 fixed points, so the lift above is in the first place a fixed-point lift. Whether it survives to the events that actually determine attack cost is a separate question, and we settled it with a preregistered paired campaign (`campaign_screening.py`, seed 20260810). The run manifest in `data_campaign_screening.json` records the command, seed and software versions, but the code commit is “unknown”, target digests are truncated, and the random source messages were not saved. The aggregate rows and the three hit blocks are preserved, but the campaign cannot be replayed byte for byte from the archive. We use it rather than assuming proportionality.

Design, fixed before the run. 40 independently generated targets; 10 contexts generated per target; every context scored on an identical cheap 2^{20} a_0 sample; the top 30% by score forming the SCREENED arm and an equal number drawn at random from the remainder forming the CONTROL arm; both arms then given *exactly* the same full 2^{32} exposure per context, with no early stop on a hit, so that a lucky context is never under-exposed. 240 contexts were fully attacked, 120 per arm, in 5.95 h of attack time on one H100 (screening excluded).

Counter (per context)	screened	control	lift
C1/C2 fixed points	159,545	19,624	$8.13\times$
W_9 lo-passes	114.86	33.73	$3.41\times$
W_9 hi-passes (given lo)	0.025	0	—
full 32-bit W_9	0.025	0	—
verified preimages	0.025	0	—

The conversion is sublinear. An $8.13\times$ lift in fixed points buys only a $3.41\times$ lift in lo-passes: screened contexts yield more fixed points, but each individual fixed point is *less* likely to pass the lo filter. An earlier pilot on 10 lo events suggested the conversion was proportional; that pilot was underpowered and this campaign supersedes it.

The effect is real but highly heterogeneous. A target-blocked bootstrap (20,000 resamples, seed 20260810, percentile interval, blocking on target because contexts within a target are not independent) gives a 95% interval of $[2.13\times, 5.82\times]$ on the pooled lo lift. Per target the median lift is $3.78\times$ and the geometric mean $3.80\times$, but the interquartile range (Weibull quantiles) spans $1.31\times$ to $12.35\times$: screening helped on 31 of 40 targets and *hurt* on the other 9, where it selected contexts richer in fixed points but poorer in lo-passes. A distribution-free sign test on the per-target lifts—which is insensitive to the heavy tail that makes the pooled ratio unstable—gives $p = 3.4 \times 10^{-4}$ one-sided (6.8×10^{-4} two-sided).

How the counter counts. The lo-pass counter is incremented once per converged trajectory: the four seed chains are deduplicated within a chain but not across chains, so a fixed point reached by two chains is counted twice. Duplicated fixed points are 0.034% of screened-arm trajectories (6,541 of 19,151,936; 4,053 of them in a single context) and 0.002% of control-arm trajectories (42 of 2,354,867); their lo status was not recorded. Because the lo test is a deterministic function of the fixed point (a_2, a_3) , a duplicate has the same lo status as its original, so a context can lose at most $\min(\text{duplicates}, \text{lo-passes})$ lo-passes under identity-level deduplication: 3,972 in the screened arm and 42 in control. That bounds the deduplicated pooled lift within $[2.42\times, 3.44\times]$, with point estimate $3.405\times$ if duplicated points pass the lo filter at the ordinary rate; since seed chains are constructed to be W_9 -consistent this is not guaranteed, and the campaign was not rerun with cross-chain deduplication. In the artefact

the screening scorer (`context_screening.py`) now deduplicates across chains; the solver that produced the campaign counts (`h100_extended.py`) is left as it ran, so the published counts can be regenerated exactly, and it records the unique fixed-point count separately (`fixed_points` against `fp_trajectories` per row).

What remains unmeasured. The screened arm produced 3 full 32-bit W_9 matches and 3 verified preimages against 0 in control. Under the null of equal rates each event falls in the screened arm with probability $1/2$, so all three doing so has $p = 1/8 = 0.125$ (one-sided): suggestive and *not* significant. We therefore report no lift for the full-match or preimage counters. Establishing one would require of order 10^2 GPU-hours, which we did not spend.

The selected arm represents three of ten generated contexts per target. A plug-in population estimate using the two observed strata is therefore $0.3(3/120) + 0.7(0/120) = 7.5 \times 10^{-3}$ per full context, corresponding to about 3.3 hours at the recorded 89-second four-seed rate. With no control-arm events and only three overall, this is a descriptive point estimate with very wide uncertainty, not an established mean.

Summary of the claim. Screening costs $1/4096$ of a context’s budget and delivers a trajectory-counted $3.41\times$ (target-blocked bootstrap 95% interval $[2.13, 5.82]$ before identity-level correction) improvement in lo-pass yield per attacked context; the available row aggregates bound the identity-deduplicated point within $[2.42, 3.44]$. It is **not** the $12.99\times$ figure quoted for fixed points, which we retain above only as a statement about fixed points; and it is not yet established as a speedup in time-to-preimage.

6 Experimental Results

6.1 Hardware and Software

All GPU experiments were run on NVIDIA H100 SXM5 80 GB HBM3 via RunPod cloud compute; the E1 and E2 pilots ran on CPU. The attack code uses PyTorch 2.x on CUDA, with the C0 sweep vectorised in batches of 2^{24} elements. The 16 GiB σ_0 -difference table is held in GPU memory throughout the sweep; table lookups are implemented as indexed tensor reads.

6.2 Verified Preimages

Each row of Table 2 was forward-verified by the following procedure:

1. Parse the 16 words $W_0, \dots, W_{15}$ as a 512-bit block.
2. Run the SHA-256 compression function for 19 rounds from the standard IV, with the standard message schedule.
3. Add the final state to the IV word-by-word.
4. Confirm the resulting 32-byte value equals the target hash.

All seven preimages pass this check. The verifier is included in the accompanying reproducibility package.

6.3 Chi-Squared Uniformity Test

To test the $1/2^{16}$ model for the W_9 -hi filter, we collected the values $(W_9^{\text{conv}} \gg 16) \bmod 2^{16}$ for all lo-pass events across 165 GPU contexts (a total of 5,775 lo-pass events). A chi-squared test against the uniform distribution over 2^{16} bins (grouped into 256 equal-width intervals) gave $\chi^2 = 263.4$ with 255 degrees of freedom ($p = 0.35$).

This test is *not* sensitive to the effect that matters, and we record the point because it is easy to be misled by it. The enhancement is concentrated at and near the exact 16-bit residual,

Table 2: Verified 19-round preimages. P1–P3 were found in dedicated runs ($K_{\text{seeds}}=0$); P4–P6 in the screened arm of the campaign of §5.7 ($K_{\text{seeds}}=4$; target and context number given). P7 is a preimage of the all-ones digest, the target of Zaikin [14] (§7), found in a dedicated run with $K_{\text{seeds}}=4$. Only the target digest was supplied to the solver; the six generated targets are 19-round digests of random 55-byte messages, so a preimage was known to exist, whereas the all-ones digest is the structured target of [14], not known in advance to have one. All values are hexadecimal. Context numbers are the solver’s zero-based indices. [†]Found 6s into the fourth context (index 3). [‡]Found 20s into the 84th context; 7,373s is the whole run.

	Hash / words				Time / note
P1	1e65261c	54255188	604f5375	09183973	871.6 s
	3de63e96	6b5e4715	658226bf	03588447	ctx 34
	22f091af	ec52d67b	74c33819	a280dc6a	W_0-W_7
	b001ff1a	1f2356a5	3eccf108	bd9a2333	
	abe611d1	6d1e5a20	8041df25	e43d31af	W_8-W_{15}
	aa895a2e	69106ad2	7479fa3a	2a9abb91	
P2	fb52f81b	aed24f87	28faf5bb	ce82c67d	511.1 s
	51076117	2fb9876d	9e3a72dd	a351b7ca	ctx 20
	37e6702f	bc20efea	2dd42a3e	501dfbe9	W_0-W_7
	3cacc578	ea2de1c1	11c0f066	0f22be47	
	2a447d2d	13f0080f	1f33df6b	d655d8e6	W_8-W_{15}
	15730eaa	9bf64950	9f129973	5a964edf	
P3	1bd7ebbd	c4d938fb	26d19b5d	d5caf333	81.7 s
	de397bd1	c745727b	d5556baf	38ccf977	ctx 3 [†]
	3ce8fba4	e2fb9661	44730c59	e1cf4bc0	W_0-W_7
	e1a18d93	97658983	67efe2a7	ef260ecb	
	d4c6dbe0	13e9388e	95664a59	4d9e248b	W_8-W_{15}
	74137862	664815ac	89eae95a	cd7dbef5	
P4	c55aecaf	d68ac4f5	59bb2b94	0688f3bc	campaign
	1dabe3ff	86a7d034	47e8b11a	637d7506	target 2, ctx 9
	3694ee67	a1d020b9	47f08ea3	dc873e2b	W_0-W_7
	1f41c8b1	39018970	d3a2afb2	23f6e6cf	
	80c46545	f7b2b46c	2e44f333	2effe68b	W_8-W_{15}
	06d80cdf	8525e7d7	7ef008e5	13e56087	
P5	a8241289	b5980304	e9dfd401	b1370200	campaign
	ab32b5e8	cfaff26c	26aabcf3	8be89df4	target 20, ctx 7
	3e0a7d67	df0944a3	4bfe156b	bd344ac5	W_0-W_7
	7b7d9587	8984e286	21c995d9	d8e53cdd	
	0a0be79d	e02dda1a	e17e0f4a	e3ff692b	W_8-W_{15}
	8c782631	97dc4811	5ed68656	dfce314f	
P6	09156002	031fed28	99bc67d0	e419b6ff	campaign
	81b9a0dd	4e3fa417	e000817a	db85b143	target 25, ctx 4
	164084a5	f97bd8bd	bff47431	a8e55b29	W_0-W_7
	948d1160	10eb8f7d	d0ea565e	7918b8b9	
	87527a3a	6baa5685	a1fbc5dc	7548adfc	W_8-W_{15}
	be3fb206	b9738756	b4ca33e3	b8354a95	
P7	ffffffff	ffffffff	ffffffff	ffffffff	7,373 s
	ffffffff	ffffffff	ffffffff	ffffffff	ctx 83 [‡]
	3c6ff45c	a5b23f8d	dc6b8089	4f232935	W_0-W_7
	79578e14	9c2da339	b22ec37a	9554191b	
	e69661f0	49a43b7a	d8c60a2b	88d7588c	W_8-W_{15}
	6225084d	87c95c4e	23ee573d	8d8fedfa	

whereas grouping into 256 equal-width intervals spreads it across a bin of width 256. We measured the dilution directly on the *lo* residual, where the exact-value enhancement is known: a $37\times$ enhancement at the exact residual appears as $3.9\times$ at 256-bin resolution, a dilution of $9.5\times$. A hi enhancement of $13\times$ diluted by the same factor would appear as $1.4\times$, shifting the occupancy of one bin from 22.6 to roughly 31—invisible to a χ^2 statistic over 255 degrees of freedom.

Measuring at exact resolution instead shows the hi filter passing at $7\text{--}13\times$ the uniform rate (E1 and E2, §5.6). The correct conclusion is that the coarse-binned test is underpowered, not that the hi residuals are uniform; filter rates in this attack must be estimated at exact residual resolution.

7 Comparison with Prior Work

Table 3: Preimage results on step-reduced SHA-256. “Concrete” means at least one preimage was computed and verified.

Reference	Rounds	Cost	Target	Concrete
Isobe–Shibutani [4]	24	2^{240}	any	no
Aoki et al. [5]	43	$2^{254.9}$	any	no
Khovratovich et al. [7]	45	$2^{255.5}$	any	no
Guo et al. [6]	47	$2^{255.6}$	any	no
Davydov et al. [13]	17, 18	SAT, seconds	zero digest	yes
Davydov et al. [13]	19	SAT, unsolved ^a	zero digest	partial
Zaikin [14]	19	18 h 33 min on 192 CPU cores ^b	all-ones digest	yes (one)
This work	19	mixed campaign: 3/240 at 89 s per attempt ^c	generated digests (six); all-ones	seven

^a Reached only in a weakened form that constrains at most 31 of the 256 output bits. ^b One successful run. Parameterized Kissat inside Cube-and-Conquer, via an intermediate problem between rounds 18 and 19 whose solutions are 19-round preimages with probability 1/8 by his account; that acceptance probability alone does not determine a mean runtime. ^c Screened arm 3/120, control arm 0/120. Pooling gives about two GPU-hours per observed event, but is not a random-context estimate; the population-weighted plug-in estimate is about 3.3 hours and is highly uncertain. The six generated targets are image-derived.

The theoretical attacks reach far more rounds than any practical method, at costs within a few bits of the generic 2^{256} , and have not been executed. The practical line is SAT-based. Davydov et al. [13] use three encoding tools, CBMC, Transalg and SAT-encoding, the last in its Counter-chain, Dot-matrix, Two-operand and Espresso adder variants, six CNF encodings in all, with the Kissat solver at a 5000 s limit, find preimages of the 17- and 18-round compression function (18 rounds in 2.38–20.6 s depending on encoding), and report 19 rounds as not solved for every encoding tried. Their weakened 19-round problem constrains only a zero prefix of the output: plain Kissat reached a 24-bit prefix, and a 24-hour cube-and-conquer run on 16 cores, applied to eight of the remaining instances, solved all of them except the 32-bit one, i.e. at most 31 of the 256 output bits. The authors conclude that finding a 19-round preimage “is too difficult a task even if the SAT encoding is carefully chosen and parallel computing is used” (our translation; the article is in Russian with an English abstract). Their published benchmark instances fix the all-zero digest as the target.

Zaikin [14] inverted the 19-round compression function first, for the all-ones digest with all 256 output bits fixed. His method constructs weakened intermediate problems between consecutive round counts, tunes Kissat on them with ParamILS, and runs a Cube-and-Conquer

solver with the tuned Kissat. Direct attempts on the 19-round instance found nothing in seven days on 192 cores; the preimage came from the intermediate problem between rounds 18 and 19 that keeps 29 of the 32 bits of the message word entering round 19 and fixes the other three to constants, whose solutions are full 19-round preimages with probability $1/8$, after 18 h 33 min on 192 CPU cores, about 3,560 core-hours (his Tables 8 and 9), against 25 s and 1 s on one core for 18 and 17 rounds. Eight off-the-shelf parallel SAT solvers, seven from the SAT Competition 2024 and ALIAS, found nothing. Zaikin’s 18 h 33 min is one successful duration; neither it nor the stated $1/8$ acceptance probability alone establishes a repeated-run mean. The attack model is the same as ours (his published preimage, Table 7 of [14], verifies under `verify_r19.py` at 19 rounds). What the present work adds is a different method and seven published witnesses. After a reusable 16-GiB precomputation, one 2^{32} attempt took about 25 seconds with a single seed chain and 89 seconds with the four chains used in the campaign. That campaign gave 3/120 successes in its selected arm and 0/120 in its control arm. The pooled two-hour figure and the population-weighted 3.3-hour plug-in figure describe that small stratified sample; neither is a precise mean. We do not convert between GPU-hours and CPU core-hours.

Head-to-head on the same target. After reading [14] we ran the corrected solver on its fixed all-ones target with the four-seed campaign configuration. It found P7 20 seconds into its 84th context, after 7,373 seconds (122.9 minutes) of solving on one H100. P7 differs from Zaikin’s Table 7 block and is therefore a second preimage of that reduced-round target. The complete run is archived in `data_ones_target_run.txt`; an earlier aborted debugging run is not included in the rate calculation.

Attack scope. The attack covers 19 of 64 rounds and targets the compression function with the standard chaining value; the padded hash is discussed in §1. Full SHA-256 is not threatened, and nothing here bears on its security margin.

8 Conclusion

Three cancellation identities in the SHA-256 round and schedule equations, together with a provisional assignment for the part of W_9 that depends on a_2, a_3 , reduce the recovery of a 19-round preimage to one 2^{32} sweep of table lookups per attempt. Six preimages of generated digests and one of the all-ones digest attacked by Zaikin [14] were found and forward-verified. A preregistered stratified campaign then recorded three successes in 120 screened attempts and none in 120 controls. The pooled value 3/240 is a description of that campaign, not a random-context success rate; the population-weighted plug-in estimate is about 3.3 GPU-hours per event and has substantial sampling uncertainty. A cheap screening pass improved the trajectory-counted intermediate yield by $3.41\times$ (identity-level deduplication bounds it between $2.42\times$ and $3.44\times$). The campaign does not establish that this intermediate improvement reduces final-preimage time.

The later context-shaped construction [1] removes the fixed-point iteration and supersedes this paper’s performance figures. It also reports computed 20-round examples under an explicitly conditional model for the extra schedule constraint. The earlier barrier note [2] records why the finite representations and update orders tested there did not absorb that constraint; it is not an impossibility result for other decompositions.

Artefacts. Solver, table construction, campaign driver, raw campaign data and an standalone verifier are available at <https://github.com/kamb-code/sha256-r19-preimage>. Any of the seven preimages (Table 2) can be checked with

```
python3 code/verify_r19.py --rounds 19 --hash <hex> --words "<W0 ...W15>"
```


A An Exact Cancellation in the Message Schedule

The σ_0 analysis above also exposes a two-word message difference that the schedule cancels exactly for eight consecutive positions.

Lemma 1 (Nine-step cancellation). *Let $W_0, \dots, W_{15}$ be any message block and let $\delta \neq 0$. Define $W'_i = W_i$ for $i \notin \{0, 9\}$, $W'_0 = W_0 + \delta$ and $W'_9 = W_9 - \delta$ (all arithmetic mod 2^{32}). Then the extended schedules satisfy $W'_r - W_r = 0$ for $r = 16, \dots, 23$, and $W'_{25} - W_{25} = -\delta$, for every block and every δ . At $r = 24$ the term $\sigma_0(W_9)$ breaks the cancellation and $W'_{24} - W_{24} = \sigma_0(W_9 - \delta) - \sigma_0(W_9)$ is nonzero in general.*

Proof. The schedule recurrence is $W_r = \sigma_1(W_{r-2}) + W_{r-7} + \sigma_0(W_{r-15}) + W_{r-16}$. For $r \in [16, 23]$ the term $\sigma_0(W_{r-15})$ indexes $r - 15 \in [1, 8]$, where the difference is zero; the term $\sigma_1(W_{r-2})$ indexes $r - 2 \in [14, 21]$, where the difference is zero up to $r = 23$ by induction; the term W_{r-7} indexes $r - 7 \in [9, 16]$ and contributes $-\delta$ only at $r = 16$; and the term W_{r-16} indexes $r - 16 \in [0, 7]$ and contributes $+\delta$ only at $r = 16$. At $r = 16$ the two contributions cancel, and for $r \in [17, 23]$ all four terms contribute zero. At $r = 25$ the only nonzero term is $W_{r-16} = W_9$, giving $-\delta$. $\square$

An exhaustive check over all 240 ordered pairs of positions in $[0, 15]$ shows that $(0, 9)$ and $(9, 0)$ are the only pairs for which ΔW_r vanishes identically for all $r \in [16, 23]$ (script `alt_differential.py`). The reason is that $\sigma_0(W_k)$ first enters the schedule at W_{k+15} , so a perturbed position $k \in [1, 8]$ seeds a nonzero difference inside the window, and cancellation of the two linear terms at $r = 16$ then requires exactly the pair $\{0, 9\}$.

Why no search result is claimed. GPU sweeps built on this difference located message pairs with one equal output word of the full 64-round compression function, and multi-block coordinate descent reached output differences of Hamming weight 72/256. We record these only to state why they are not results. For a fixed input difference, the expected minimum Hamming weight of a random 256-bit output difference over 2^{32} independent trials is 78 (median 78), and over the 2^{37-239} evaluations used by the multi-block searches it is 74–73; a minimum of 72 or below occurs by chance with probability 0.11–0.38 over that many independent trials, and a directed descent does at least as well. The observed minima (75–95 over single 2^{32} sweeps, 72 after multi-block descent) are therefore at the level expected of an unbiased difference, and one-word equalities occurred at the uniform rate of 2^{-32} per output word per trial. The code and data remain in the repository for completeness.

Note added, September 2026. After this paper was written, a context-shaped variant [1] imposed $a_4 = a_5$ and, for the derived words, $e_8 = e_9 = 0xFFFFFFFF$. Those conditions remove the fixed-point iteration and collapse the consistency check to a bitwise condition. The observed 19-round cost falls to about $2^{15.25}$ swept a_0 values. At 20 rounds a fourth exact equality remains; treating its measured residual as uniform at exact zero gives the paper’s conditional estimate of about $2^{45.4}$ swept values and about 15 A100-hours. Two computed 20-round examples are reported there, including one for the all-ones digest. That paper supersedes the cost estimates here. The campaign measurements and screening result in this paper remain evidence about the original construction, but the small stratified campaign does not provide a precise random-context mean.

Acknowledgements and disclosure

The derivations, code, experiments and original text were developed with substantial assistance from Anthropic’s Claude, used through Claude Code under the author’s direction. OpenAI

Codex assisted with this revision by auditing claims against the archived records, checking references, editing the manuscript and rebuilding the publication files. The repository also records separate AI-agent verification passes. Three of those passes implemented the reduced-round compression function separately from the specification and forward-checked supplied witnesses; other passes reviewed shared source, manifests or logs. They are automated checks, not human peer review. The author selected the claims and remains responsible for the manuscript.

References

- [1] K. S. Basra. Context shaping for 19- and 20-round SHA-256 compression preimages. Manuscript, 2026. Source, code and witnesses at <https://github.com/kamb-code/sha256-r19-preimage>.
- [2] K. S. Basra. The 20-round barrier for table-based preimage attacks on SHA-256. Companion note, 2026. Source and code in the same repository.
- [3] National Institute of Standards and Technology. *FIPS PUB 180-4: Secure Hash Standard (SHS)*, 2015.
- [4] T. Isobe and K. Shibutani. Preimage attacks on reduced Tiger and SHA-2. In *Fast Software Encryption (FSE 2009)*, LNCS 5665, pp. 139–155. Springer, 2009.
- [5] K. Aoki, J. Guo, K. Matusiewicz, Y. Sasaki, and L. Wang. Preimages for step-reduced SHA-2. In *Advances in Cryptology – ASIACRYPT 2009*, LNCS 5912, pp. 578–597. Springer, 2009.
- [6] J. Guo, H. Li, M. Liu, S. Wang, and T. Zhang. Dual-syncopation meet-in-the-middle attacks: new results on SHA-2 and MD5. In *Advances in Cryptology – EUROCRYPT 2026*, LNCS 16546, pp. 121–151. Springer, 2026. DOI 10.1007/978-3-032-25333-0_5. Cryptology ePrint Archive, Report 2026/353.
- [7] D. Khovratovich, C. Rechberger, and A. Savelieva. Bicliques for preimages: attacks on Skein-512 and the SHA-2 family. In *Fast Software Encryption (FSE 2012)*, LNCS 7549, pp. 244–263. Springer, 2012.
- [8] S. K. Sanadhya and P. Sarkar. New collision attacks against up to 24-step SHA-2. In *Progress in Cryptology – INDOCRYPT 2008*, LNCS 5365, pp. 91–103. Springer, 2008.
- [9] F. Mendel, T. Nad, and M. Schl  ffer. Improving local collisions: new attacks on reduced SHA-256. In *Advances in Cryptology – EUROCRYPT 2013*, LNCS 7881, pp. 262–278. Springer, 2013.
- [10] Y. Li, F. Liu, and G. Wang. New records in collision attacks on SHA-2. In *Advances in Cryptology – EUROCRYPT 2024, Part I*, LNCS 14651, pp. 158–186. Springer, 2024. DOI 10.1007/978-3-031-58716-0_6.
- [11] Y. Li, F. Liu, G. Wang, X. Dong, and S. Sun. The first practical collision for 31-step SHA-256. In *Advances in Cryptology – ASIACRYPT 2024, Part VII*, LNCS 15490, pp. 237–266. Springer, 2024. DOI 10.1007/978-981-96-0941-3_8.
- [12] I. Mironov and L. Zhang. Applications of SAT solvers to cryptanalysis of hash functions. In *Theory and Applications of Satisfiability Testing (SAT 2006)*, LNCS 4121, pp. 102–115. Springer, 2006.

- [13] V. V. Davydov, M. D. Pikhtovnikov, A. P. Kiryanova, and O. S. Zaikin. Analysis of the cryptographic strength of the SHA-256 hash function using the SAT approach. *Scientific and Technical Journal of Information Technologies, Mechanics and Optics*, 25(3):428–437, 2025. DOI 10.17586/2226-1494-2025-25-3-428-437.
- [14] O. Zaikin. Preimage attacks on round-reduced MD5, SHA-1, and SHA-256 using parameterized SAT solver. *Constraints*, 2026 (received 28 March 2025, accepted 16 December 2025, published online 23 January 2026). DOI 10.1007/s10601-025-09383-0.